

Edmonds-Karp Algorithm

Shortest Augmenting Paths and Polynomial-Time Max Flow

1. What is the Edmonds-Karp Algorithm?

The **Edmonds-Karp algorithm** is an implementation of the Ford-Fulkerson method for computing the maximum flow in an optimistic network. The revolutionary difference is its use of **Breadth-First Search (BFS)** instead of DFS.

Core Advantage: By using BFS, Edmonds-Karp always finds the *shortest augmenting path* (in terms of number of edges), which guarantees polynomial time complexity regardless of the edge capacities.

2. BFS vs. DFS: Why it Matters

In standard Ford-Fulkerson (DFS), the algorithm can be tricked into taking extremely inefficient paths if the capacities are high. Edmonds-Karp avoids this "Capacity Sensitivity" by prioritizing paths with the fewest edges.

- **DFS (Ford-Fulkerson):** Might iterate 10,000 times for a max flow of 10,000.
- **BFS (Edmonds-Karp):** Focuses on topological distance, limiting the number of total augmentations.

3. The Parent Array Mechanism

While traversing via BFS, Edmonds-Karp maintains a `parent[]` array. This array is essential for backtracking from the **Sink (t)** to the **Source (s)** once a path is discovered, allowing the algorithm to identify the bottleneck and update residual capacities.

```
# BFS exploring the level graph
while queue:
    u = queue.pop(0)
    for v, capacity in enumerate(self.adj[u]):
        if not visited[v] and capacity > 0:
            parent[v] = u
            visited[v] = True
            queue.append(v)
```

4. Time Complexity: $O(V * E^2)$

The time complexity is independent of the maximum flow value (f), making it robust for graphs with large capacities:

Complexity: $O(V * E^2)$.

- Each BFS takes $O(E)$.
- The number of total augmentations is bounded by $O(V * E)$.

5. Python Implementation

```
def edmonds_karp(self, s, t):
    parent = [-1] * self.size
    max_flow = 0
    while self.bfs(s, t, parent): # BFS search
        path_flow = float("Inf")
```

```
    curr = t
    while curr != s: # Find bottleneck
        path_flow = min(path_flow, self.adj[parent[curr]][curr])
        curr = parent[curr]

    max_flow += path_flow
    v = t
    while v != s: # Update residual graph
        u = parent[v]
        self.adj[u][v] -= path_flow
        self.adj[v][u] += path_flow
        v = parent[v]
    return max_flow
```

CodeHive

Theoretical Efficiency Analysis (Part 1)

The "Shortest Augmenting Path" heuristic is the genius of Edmonds-Karp. In a network where some edges have massive capacities and others have tiny ones, a DFS-based approach might choose a long path that increases flow by only 1 unit. By using BFS, Edmonds-Karp ensures we move through the **Layered Network**, systematically saturating edges.

Note on Space Complexity: The algorithm requires $O(V^2)$ space for an Adjacency Matrix or $O(V+E)$ for an Adjacency List. In modern software engineering, Adjacency Lists are preferred for sparse real-world networks.

Convergence Guarantee

Unlike Ford-Fulkerson, which can fail to terminate on graphs with irrational edge capacities, Edmonds-Karp is guaranteed to terminate because the distance to any node in the residual network never decreases during an augmentation. This monotonic property is the foundation of its stability.

Theoretical Efficiency Analysis (Part 2)

The "Shortest Augmenting Path" heuristic is the genius of Edmonds-Karp. In a network where some edges have massive capacities and others have tiny ones, a DFS-based approach might choose a long path that increases flow by only 1 unit. By using BFS, Edmonds-Karp ensures we move through the **Layered Network**, systematically saturating edges.

Note on Space Complexity: The algorithm requires $O(V^2)$ space for an Adjacency Matrix or $O(V+E)$ for an Adjacency List. In modern software engineering, Adjacency Lists are preferred for sparse real-world networks.

Convergence Guarantee

Unlike Ford-Fulkerson, which can fail to terminate on graphs with irrational edge capacities, Edmonds-Karp is guaranteed to terminate because the distance to any node in the residual network never decreases during an augmentation. This monotonic property is the foundation of its stability.

Theoretical Efficiency Analysis (Part 3)

The "Shortest Augmenting Path" heuristic is the genius of Edmonds-Karp. In a network where some edges have massive capacities and others have tiny ones, a DFS-based approach might choose a long path that increases flow by only 1 unit. By using BFS, Edmonds-Karp ensures we move through the **Layered Network**, systematically saturating edges.

Note on Space Complexity: The algorithm requires $O(V^2)$ space for an Adjacency Matrix or $O(V+E)$ for an Adjacency List. In modern software engineering, Adjacency Lists are preferred for sparse real-world networks.

Convergence Guarantee

Unlike Ford-Fulkerson, which can fail to terminate on graphs with irrational edge capacities, Edmonds-Karp is guaranteed to terminate because the distance to any node in the residual network never decreases during an augmentation. This monotonic property is the foundation of its stability.